implementations, you can create complex authorization policies that meet any of your business requirements.

For a policy to be satisfied, a user must fulfill all the requirements. If the user fails any of the requirements, the authorize middleware won't allow the protected endpoint to be executed. In this example, a user must be allowed to access the lounge *and* must be over 18 years old.

Each requirement can have one or more handlers, which will confirm that the requirement has been satisfied. For example, as shown in figure 24.5, `AllowedInLoungeRequirement` has two handlers that can satisfy the requirement:

- `FrequentFlyerHandler`
- `IsAirlineEmployeeHandler`

If the user satisfies either of these handlers, `AllowedInLoungeRequirement` is satisfied. You don't need all handlers for a requirement to be satisfied; you need only one.

NOTE Figure 24.5 shows a third handler, `BannedFromLoungeHandler`, which I'll cover in section 24.4.2. It's slightly different in that it can fail a requirement but not satisfy it.

You can use requirements and handlers to achieve most any combination of behavior you need for a policy. By combining handlers for a requirement, you can validate conditions using a logical `OR`: if any of the handlers is satisfied, the requirement is satisfied. By combining requirements, you create a logical `AND`: all the requirements must be satisfied for the policy to be satisfied, as shown in figure 24.6.